# Toward a Systematic Evaluation of Usability in RESTful APIs: A Metric-Based Approach

*Appendix*

Ariel Machini

amachini (AT) conicet (DOT) gov (DOT) ar

March 2026

**Abstract**

This document contains complementary content (mostly figures and tables) that could not be included in the main article due to space constraints.

# 1 Method

This Section provides additional information related to the methodological approaches used during the development process of the activities performed.

## 1.1 The Goal-Question-Metric approach

As mentioned in the main article, the Goal-Question-Metric (GQM) approach was leveraged to structurally define the model. For context, a definition of this approach is provided below.

According to [Basili et al., 2002], "The result of the application of the Goal Question Metric approach is the specification of a measurement system targeting a particular set of issues and a set of rules for the interpretation of the measurement data". The resulting measurement model contains three different, but connected levels:

- **Conceptual level (Goals):** Goals are defined for objects, which can be products (artifacts, deliverables, and documents that are produced during the system life cycle), processes (software related activities normally associated with time) and resources (items used by processes in order to produce their outputs).
- **Operational level (Questions):** Questions are used to characterize the way the achievement of a specific goal is going to be performed. Questions try to characterize

the object of measurement with respect to a selected quality issue and to determine its quality from the selected viewpoint.

- **Quantitative level (Metrics):** A set of data is associated with every question in order to answer it in a quantitative way. The data can be objective (if they depend only on the object that is being measured) and subjective (if they depend on both the object that is being measured and the viewpoint from which they are taken).

# 2 Systematic mapping study

## 2.1 Python scripts

While conducting the systematic mapping study, it was found that Springer Link (one of the academic databases used to search for articles) did not allow searching in specific parts of an article, which in turn led to a large number of search results. Additionally, Springer Link only allowed downloading results as a CSV file. To overcome these challenges, two scripts were developed to (1) complete the entries with their corresponding abstracts (whenever possible) and (2) convert the CSV file to a BibTeX file, so it could be fed to JabRef to further narrow the number of results.

### 2.1.1 Springer Abstract Scraper

The source code for this Python script (which is based on the work of rohitdwivedula) is available at this GitHub gist.

### 2.1.2 Springer2BibTeX

The source code for this Python script is available at this GitHub repository.

## 2.2 Identified articles

In the main article it is mentioned that, as a result of the execution of the systematic mapping study, 18 articles of interest were identified. Table 1 lists said articles (sorted by year), and Figure 1 illustrates the filtering process followed to determine the final set of articles.

# 3 Goal-Question-Metric usability model

As mentioned in the main article, the latest version of the model is composed of six goals (corresponding to the conceptual level), eight questions (operational level), and 45 metrics

| Title | Authors | Year |
|---|---|---|
| *Improving Web API Usage Logging* | [Koçi et al., 2021] | 2021 |
| *Integration of RESTFul API to Student Information System for Secured Data Sharing and Single Sign-on* | [Serrano and Oñate, 2021] | 2021 |
| *A Data-Driven Approach to Measure the Usability of Web APIs* | [Koçi et al., 2020] | 2020 |
| *Contexts Enhance Accuracy: On Modeling Context Aware Deep Factorization Machine for Web API QoS Prediction* | [Shen et al., 2020] | 2020 |
| *A systematic mapping study of API usability evaluation methods* | [Rauf et al., 2019] | 2019 |
| *A systematic approach to API usability: Taxonomy-derived criteria and a case study* | [Mosqueira-Rey et al., 2018] | 2018 |
| *An empirical study for evaluating the performance of multi-cloud APIs* | [Ré et al., 2018] | 2018 |
| *Discovering API usability problems at scale* | [Murphy-Hill et al., 2018] | 2018 |
| *QoS-Aware Selection of Web APIs Based on $\varepsilon$-Pareto Genetic Algorithm* | [Ma et al., 2016] | 2016 |
| *Automated measurement of API usability: The API Concepts Framework* | [Scheller and Kühn, 2015] | 2015 |
| *Some structural measures of API usability* | [Rama and Kak, 2015] | 2015 |
| *A quality model for mobile thick client that utilizes web API* | [Fauzia et al., 2014] | 2014 |
| *Towards Understanding of Structural Attributes of Web APIs Using Metrics Based on API Call Responses* | [Janes et al., 2014] | 2014 |
| *An Empirical Study of API Usability* | [Piccioni et al., 2013] | 2013 |
| *Methods towards API Usability: A Structural Analysis of Usability Problem Categories* | [Grill et al., 2012] | 2012 |
| *The concept maps method as a tool to evaluate the usability of APIs* | [Gerken et al., 2011] | 2011 |
| *Useful, But Usable? Factors Affecting the Usability of APIs* | [Zibran et al., 2011] | 2011 |
| *Automatic evaluation of API usability using complexity metrics and visualizations* | [de Souza and Bentolila, 2009] | 2009 |

**Table 1:** Articles found through the systematic mapping study.

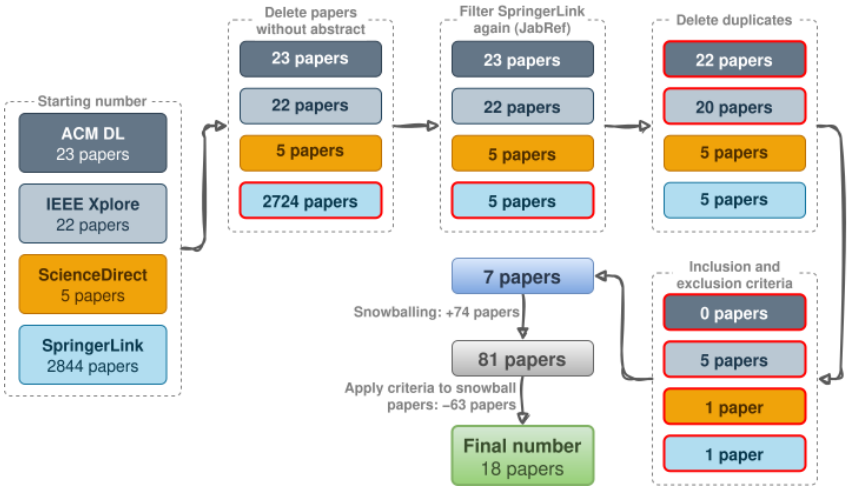


**Figure 1:** Steps of the filtering process, carried out as part of the systematic mapping study.

(quantitative level). Figure 2 presents the different levels and components of the model in a hierarchical manner.

In the context of the Goal-Question-Metric approach, a metric can be either (O)bjective or (S)ubjective. For context, an objective metric depends only on the object that is being measured, while a subjective metric also depends on the viewpoint from which it is taken [Basili et al., 2002].

## 3.1 Metrics

Table 2 details all metrics related to the quantitative level of the proposed model.

| Metric | Description |
|---|---|
| **Abstraction level** | Abstraction level exposed by the web API. Leaking implementation details can cause confusion for third-party developers and lead to a bad experience. |
| **Abstraction of endpoint names** | How abstract endpoint names are. Using the right level of abstraction for names helps developers understand what they can do with the web API. |
| **Abstraction of parameter names** | How abstract parameter names are. Using the right level of abstraction for names helps developers understand what they can do with the web API. |
| **Average base URLs per resource** | Average number of base URLs per resource. A simple and intuitive base URL design makes a web API easy to use. |
| **Average number of parameters** | Average number of parameters endpoints have. Long lists of parameters should be avoided when possible. |
| **Average response time** | Time elapsed from the moment the developer makes a request until it receives a response back. High-performing APIs are considered to have between 0.1 and 1 second average response time. At 2 seconds, the delay is noticeable. At 5 seconds, the delay can cause the developers to abandon the API. |
| **Clarity of endpoint names** | How clear endpoint names are. Using concise and self-explanatory endpoint names improves the understandability of a web API. Code names, awkward vocabulary, cryptic abbreviations, and having more than three words should be avoided for endpoint names. |

| | |
|---|---|
| **Complexity of returned data** | How complex the data returned by the endpoints that are meant to return something are. Using appropriate data formats for returned data is important to provide a straightforward representation. Avoid overly complex or cryptic data formats. For JSON objects, a maximum depth of 3 and a maximum number of properties of 20 is recommended. |
| **Consistency of endpoint names** | How consistent endpoint names are. Language and conventions used for naming endpoints should be consistent. A consistent design is easy to understand and use. |
| **Consistency of parameter names** | How consistent parameter names are. Language and conventions used for naming parameters should be consistent. A consistent design is easy to understand and use. |
| **Consistency of parameter types** | Whether or not there are parameters with the same name but different types on the web API. Having parameters with identical names but distinct types across different endpoints leads to code bloat in developers' code bases, which negatively impacts usability. |
| **Consistency of returned data** | How consistent the data returned by the web API are. Language and conventions used for formatting returned data should be consistent. A consistent design is easy to understand and use. |
| **Developer stack size** | Number of additional tools and libraries a developer has to install in order to use the web API. |
| **Documentation of endpoints** | Proportion of endpoints that are documented. Ideally, every element of the web API should be documented. |
| **Documentation of HTTP methods** | Proportion of HTTP methods (part of the endpoints) that are documented. Ideally, every element of the web API should be documented. |
| **Endpoint grouping by functionality** | Whether or not endpoints with similar functionality are visibly grouped. Grouping endpoints by functionality facilitates navigating through the API and improves understandability. This can be achieved by sorting endpoints in the definition document and leveraging the API specification format. |
| **Endpoint name-functionality correspondence** | Whether or not the endpoints of the web API perform the tasks suggested by their name. Endpoints should not do less/more than what their names suggest and, if possible, not more than one thing. |
| **Error categorization by type** | Whether or not errors returned by the web API are grouped or categorized by their meaning. Organizing feedback facilitates interpretation, makes troubleshooting easier and reduces developer frustration. |
| **Identification of deprecated elements** | Whether or not the documentation of the web API includes deprecated elements. Deprecated elements should be clearly identified. |
| **Identification of required parameters** | Whether or not all required endpoint parameters are explicitly marked as "required". Not doing so might cause unexpected errors and frustration for developers. |
| **Identification of required properties in responses** | Whether or not all required properties in responses are explicitly marked as "required". Not doing so might cause unexpected errors and frustration for developers. |
| **Inclusion of a getting started guide** | Whether or not the documentation of the web API includes a getting started guide. Having a getting started guide in the documentation helps users to get up and running with the API faster. |
| **Inclusion of authentication information** | Whether or not the documentation of the web API includes information about authentication (if applicable). Almost every API features some sort of authentication schema, and nearly every one is different. It is because of this that the API documentation should have information on how to access the web API. |
| **Inclusion of change information** | Whether or not the documentation of the web API includes a changelog section. Having a changelog section in the documentation lets users know how stable the API is. It also lets them know if anything has changed, in case that one of their calls stops working. |
| **Inclusion of error information** | Whether or not the documentation of the web API includes error information. Ideally, the API documentation should include information about common and important errors. |

| | |
|---|---|
| **Inclusion of usage examples** | Whether or not the documentation of the web API includes usage examples. Ideally, the API documentation should include code samples for the most common scenarios. |
| **JSON support** | Whether or not the REST API supports the JSON type in requests and/or responses. |
| **Number of HTTP 4XX errors in a period of time** | Number of client-side errors that occur in a specified period of time. If the number of errors is high with respect to the total number of API calls made in the defined period, documentation might be deficient, naming choices might be poor, or something along these lines may be causing confusion for the web API users. |
| **Number of HTTP 5XX errors in a period of time** | Number of server-side errors that occurs in a specified period of time. If the number of errors is high with respect to the total number of API calls made in the defined period, the server where the API is hosted might be malfunctioning. This is critical, since it may be motivating enough for developers to look for alternative APIs. |
| **OAuth usage** | Whether or not the web API supports OAuth for authentication. OAuth is a widely known standard, and using it facilitates user experience. |
| **Proper String usage** | Whether or not the String type (or an equivalent) is used only when necessary. Strings should not be used if a better type exists, since they are cumbersome and error-prone. |
| **Proper usage of headers** | Proportion of cases where headers are used appropriately. Defining proper headers helps provide useful information about the server behavior. |
| **Similarity of endpoint names** | Whether or not there are endpoints with similar names but different functionality. In general, similar names confuse users and increase the chances of making errors. |
| **Specification usage** | Whether or not the web API provides a specification such as OpenAPI for its users. In addition to serving as a piece of documentation, a specification can help the developer with error checking and validation. |
| **Usage of components (OpenAPI)** | Whether or not the OpenAPI spec of the web API uses components. Using components makes the OpenAPI specification more readable and removes unnecessary duplication. Components provide a powerful mechanism for defining reusable schemas, parameters, responses, and other elements within the specification. |
| **Usage of formats and/or patterns (OpenAPI)** | Whether or not the OpenAPI spec of the web API uses formats and/or patterns. Using formats and/or patterns within the OpenAPI spec helps users better understand the kind of values that are expected for a given parameter and add an extra step of validation. Some examples are "format: uuid" and "format: email" and, if there is no predefined format that suits the data represented by a parameter, the "pattern" property can be used to define a regular expression that adapts to it. Note that using formats and/or patterns is not really necessary for simple fields, as this would add unneeded complexity. |
| **Specificity of parameter types** | Whether or not all types of endpoint parameters are as specific as possible. Data types should be as specific as possible to make the code more readable. |
| **Specificity of status codes** | How specific status codes returned by the web API are. Ideally, status codes should be as specific as possible, as this could be valuable information for developers and could increase understandability. |
| **Support for caching** | Whether or not the web API supports caching. In some cases, caching can increase the API performance and, therefore, improve usability. If caching is supported, it should be specified with the Cache-Control header. |
| **Support for data filtering, pagination and sorting** | Whether or not the web API supports data filtering, pagination and sorting for returned data. Providing developers with these tools improves usability and helps make the web API more adaptable. |
| **Support for multiple return formats** | Whether or not the web API supports multiple return formats. This helps make the web API more adaptable and improves usability. |

| | |
|---|---|
| **Usage of plural nouns for resource names** | Whether or not resource (endpoint) names are plural when applicable. The use of singular nouns does not give an idea of whether there are multiple items within, leading to confusion, and having both singular and plural nouns will double the API requests list. |
| **Usage of verbs in base URLs** | Whether or not the web API has endpoints with method-based names. Verbs on endpoint names should be avoided, as they do not convey any new information about the request, since this is handled by the HTTP methods associated with REST. |
| **Verbosity of error messages** | Verbosity level of error messages returned by the web API. Error feedback must be as informative as possible, so there is no room for confusion. |
| **Versioning** | Whether or not the web API is versioned. Versioning enables breaking changes to be made with new versions while maintaining backward compatibility for old versions. |

**Table 2:** Final set of metrics related to the quantitative level of the Goal-Question-Metric usability model. These metrics are also used for the catalog.

## 3.2 Automation proof of concept

In the main article, more specifically, in Section *Conclusions and future work*, it is pointed out that one of the primary objectives for future work is automating metric assessment (when possible) with a tool that is currently being developed. In this Section, a proof of concept with a single metric (Average base URLs per resource) is provided to exemplify how the automation process will work. Figure 3 displays a flowchart summarizing the logic behind the assessment of this particular metric.

Additionally, Figure 4 presents the pseudocode corresponding to the function that automates the assessment of the aforementioned metric.

# 4 Gwet's AC1 coefficient

In the main article, it is stated that "[s]tatistical analysis of the responses showed substantial agreement". For clarification, the word *substantial* was taken from [Landis and Koch, 1977] and [Wongpakaran et al., 2013], where the authors indicate that a value above 0.60[1] indicates *substantial* agreement among the raters.

[1]The conducted statistical analysis returned an AC1[Gwet, 2008] value of **0.67**.

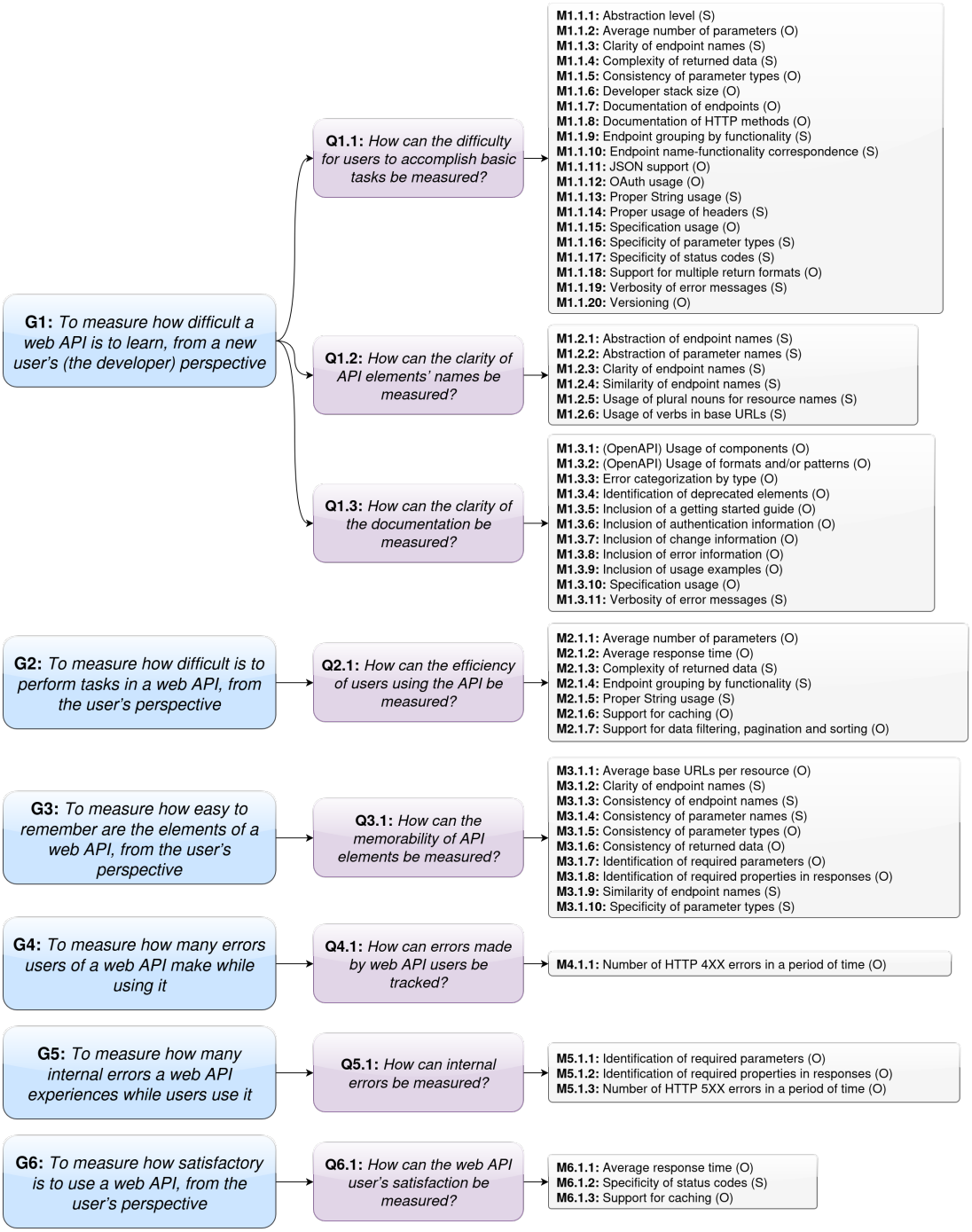


**Figure 2:** The GQM usability model.

*Disclaimer:* Please note that the numbering style used to label the metrics is subject to change.

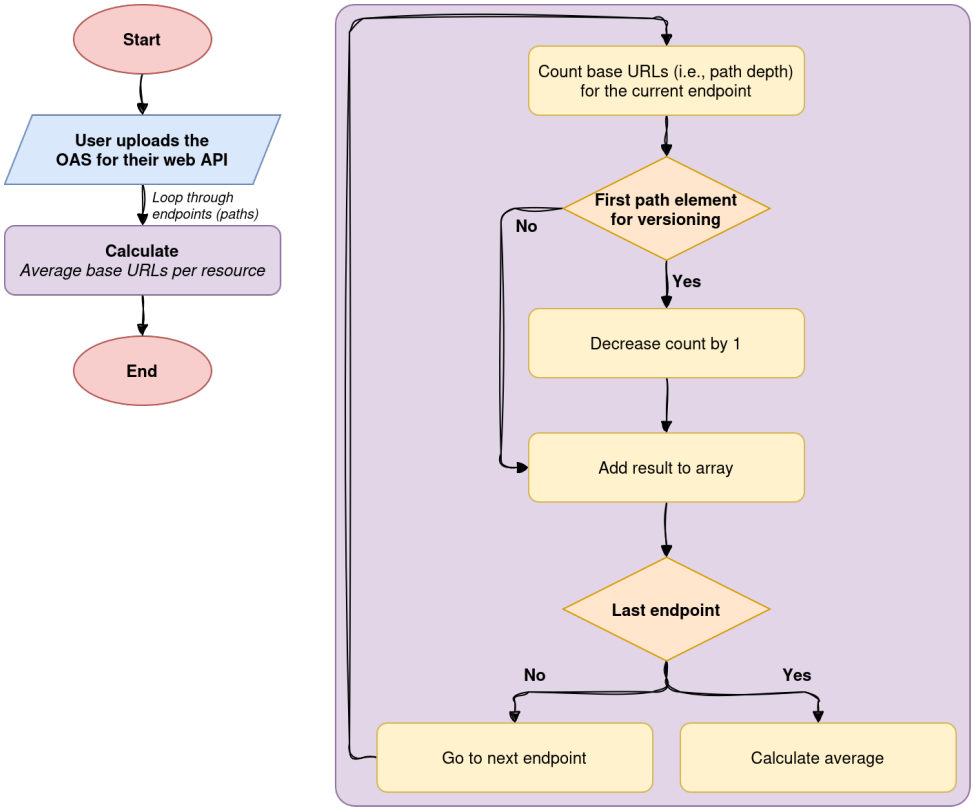


**Figure 3:** Flowchart for the assessment of "Average base URLs per resource".

**function** `assessAvgBaseURLsPerResource(OAS)` {The OpenAPI spec is user-provided.}
`pathDepthCounts` ← `Empty list`
**for all** `path` **in** `OAS.paths` **do**
  `explodedPath` ← List of `path` components, split by `/`
  `pathDepth` ← Length of `explodedPath`
  **if** First element of `explodedPath` matches versioning pattern (e.g., "v1", "V2.0") **then**
    `pathDepth` ← `pathDepth` - 1
  **end if**
  Add `pathDepth` to `pathDepthCounts`
**end for**
`avgPathDepth` ← Sum of `pathDepthCounts` / Length of `pathDepthCounts`
**if** `avgPathDepth` $\leq 2$ **then**
  `assessmentResult` ← 1
**else if** `avgPathDepth` $== 3$ **then**
  `assessmentResult` ← 0.5
**else**
  `assessmentResult` ← 0
**end if**
Update metric "Average base URLs per resource" with value `assessmentResult`

**Figure 4:** Pseudocode for the automatic assessment of "Average base URLs per resource".

# References


[Basili et al., 2002] Basili, V., Caldiera, G., and Rombach, D. (2002). *Goal Question Metric (GQM) Approach*, pages 528–532. John Wiley & Sons, Ltd. This is a 2002 revision of the original version published in 1994.

[de Souza and Bentolila, 2009] de Souza, C. R. B. and Bentolila, D. L. M. (2009). Automatic evaluation of API usability using complexity metrics and visualizations. In *2009 31st International Conference on Software Engineering - Companion Volume*, pages 299–302.

[Fauzia et al., 2014] Fauzia, H., Laksmiwati, I. H., and Hendradjaya, B. (2014). A quality model for mobile thick client that utilizes web API. In *2014 International Conference on Data and Software Engineering (ICODSE)*, pages 1–6.

[Gerken et al., 2011] Gerken, J., Jetter, H.-C., Zöllner, M., Mader, M., and Reiterer, H. (2011). The concept maps method as a tool to evaluate the usability of APIs. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '11, pages 3373–3382, New York, NY, USA. Association for Computing Machinery.

[Grill et al., 2012] Grill, T., Polacek, O., and Tscheligi, M. (2012). Methods towards API Usability: A Structural Analysis of Usability Problem Categories. In Winckler, M., Forbrig, P., and Bernhaupt, R., editors, *Human-Centered Software Engineering*, pages 164–180, Berlin, Heidelberg. Springer Berlin Heidelberg.

[Gwet, 2008] Gwet, K. L. (2008). Computing inter-rater reliability and its variance in the presence of high agreement. *British Journal of Mathematical and Statistical Psychology*, 61(1):29–48.

[Janes et al., 2014] Janes, A., Remencius, T., Sillitti, A., and Succi, G. (2014). Towards Understanding of Structural Attributes of Web APIs Using Metrics Based on API Call Responses. In Corral, L., Sillitti, A., Succi, G., Vlasenko, J., and Wasserman, A. I., editors, *Open Source Software: Mobile Open Source Technologies*, pages 83–92, Berlin, Heidelberg. Springer Berlin Heidelberg.

[Koçi et al., 2020] Koçi, R., Franch, X., Jovanovic, P., and Abelló, A. (2020). A Data-Driven Approach to Measure the Usability of Web APIs. In *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 64–71.

[Koçi et al., 2021] Koçi, R., Franch, X., Jovanovic, P., and Abelló, A. (2021). Improving Web API Usage Logging. In Cherfi, S., Perini, A., and Nurcan, S., editors, *Research Challenges in Information Science*, pages 623–629, Cham. Springer International Publishing.

[Landis and Koch, 1977] Landis, J. R. and Koch, G. G. (1977). The Measurement of Observer Agreement for Categorical Data. *Biometrics*, 33(1):159–174.

[Ma et al., 2016] Ma, S.-P., Lan, C.-W., Ho, C.-T., and Ye, J.-H. (2016). QoS-Aware Selection of Web APIs Based on $\varepsilon$-Pareto Genetic Algorithm. In *2016 International Computer Symposium (ICS)*, pages 595–600.

[Mosqueira-Rey et al., 2018] Mosqueira-Rey, E., Alonso-Ríos, D., Moret-Bonillo, V., Fernández-Varela, I., and Álvarez Estévez, D. (2018). A systematic approach to API usability: Taxonomy-derived criteria and a case study. *Information and Software Technology*, 97:46–63.

[Murphy-Hill et al., 2018] Murphy-Hill, E., Sadowski, C., Head, A., Daughtry, J., Macvean, A., Jaspan, C., and Winter, C. (2018). Discovering API usability problems at scale. In *Proceedings of the 2nd International Workshop on API Usage and Evolution*, WAPI '18, pages 14–17, New York, NY, USA. Association for Computing Machinery.

[Piccioni et al., 2013] Piccioni, M., Furia, C. A., and Meyer, B. (2013). An Empirical Study of API Usability. In *2013 ACM / IEEE International Symposium on Empirical Software Engineering and Measurement*, pages 5–14.

[Rama and Kak, 2015] Rama, G. M. and Kak, A. (2015). Some structural measures of API usability. *Software: Practice and Experience*, 45(1):75–110.

[Rauf et al., 2019] Rauf, I., Troubitsyna, E., and Porres, I. (2019). A systematic mapping study of API usability evaluation methods. *Computer Science Review*, 33:49–68.

[Ré et al., 2018] Ré, R., Manciola Meloca, R., Nassif Roma, D., da Cruz Ismael, M. A., and Costa Silva, G. (2018). An empirical study for evaluating the performance of multi-cloud APIs. *Future Generation Computer Systems*, 79:726–738.

[Scheller and Kühn, 2015] Scheller, T. and Kühn, E. (2015). Automated measurement of API usability: The API Concepts Framework. *Information and Software Technology*, 61:145–162.

[Serrano and Oñate, 2021] Serrano, P. A. M. and Oñate, J. J. S. (2021). Integration of RESTFul API to Student Information System for Secured Data Sharing and Single Sign-on. In *2021 IEEE 13th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM)*, pages 1–6.

[Shen et al., 2020] Shen, L., Pan, M., Liu, L., You, D., Li, F., and Chen, Z. (2020). Contexts Enhance Accuracy: On Modeling Context Aware Deep Factorization Machine for Web API QoS Prediction. *IEEE Access*, 8:165551–165569.

[Wongpakaran et al., 2013] Wongpakaran, N., Wongpakaran, T., Wedding, D., and Gwet, K. L. (2013). A comparison of Cohen's Kappa and Gwet's AC1 when calculating inter-rater reliability coefficients: a study conducted with personality disorder samples. *BMC medical research methodology*, 13:61.

[Zibran et al., 2011] Zibran, M. F., Eishita, F. Z., and Roy, C. K. (2011). Useful, But Usable? Factors Affecting the Usability of APIs. *2011 18th Working Conference on Reverse Engineering*, pages 151–155.